

A13x Pipeline Maya Tool Suite

Setup, PyPI installation, per-line customization, and licensing
guide

Included tools: GeoMaster • Revention • Substance Loader • ThumbTac • Umbra •
Nomenclator

For end users installing and customizing the A13x tool set inside Autodesk Maya.

Quick Start — Read This First

To install A13x and have it working in Maya, run one script. That's the whole process:

OS	Do this
Windows	Double-click <code>install/install_a13x_windows.bat</code>
macOS / Linux	Run <code>./install/install_a13x_mac_linux.sh</code>

This one script finds **every** Maya version installed on the machine, installs A13x into all of them, and registers the plug-in in each — automatically, with no prompts. The **A13x** shelf then appears on its own the next time you open any of those Maya versions. Nothing else is required.

Plain `pip install a13x`, by itself, does NOT do this. `pip` only ever copies files — it cannot run code as a side effect of installing, on this package or any other. The script above works because it runs `pip install` *and then*, as a clearly separate step, registers the plug-in for you. If you ever run `pip install a13x` directly instead of using the script, you still need one more command afterward — either `a13x-register-plugin` from a terminal, or `installer.install()` from Maya's Script Editor. See Section 2.6 for exactly what each of those does.

If something doesn't work

If this happens...	...go here
"mayapy isn't recognized as an internal or external command"	Section 2.4 note — mayapy only exists inside an actual Maya install; it can't be separately installed
The shelf didn't appear after opening Maya	Section 2.6 — check Plug-in Manager's Auto load checkbox for <code>a13x_plugin.py</code>
"pip uninstall says it isn't installed"	Section 2.8 — almost always means a different Python installed it; use <code>a13x-uninstall</code> instead
Need to wipe everything and start clean	Section 2.8 — <code>a13x-uninstall</code> , or the <code>uninstall_a13x_*</code> script
Want to know exactly where files landed on disk	Section 2.5 (<code>pip install</code>) and Section 2.9 (search locations)

2.7a Initializing Script — Quick Reference

The two lines below are the entire Maya-side initializing script referenced throughout Section 2. Paste them into Maya's Script Editor (Python tab) once, after the pip install described in Section 2.4, and run:

```
import a13x.installer as installer
installer.install()
```

This registers the plug-in stub in Maya's Plug-in Manager, turns Auto load on, builds the A13x shelf, and opens this guide — see Section 2.6 for what each part of `install()` does in detail.

Contents

- 1. Overview of the Tool Suite
- 2. Setup & Installation
 - 2.1 – 2.3 Manual copy install (scripts folder)
 - 2.4 Installing via pip — TestPyPI & PyPI
 - 2.5 Where Everything Gets Installed
 - 2.6 Loading the Plug-in & Using the Shelf — Autoload Explained
 - 2.7 Opening This Guide — installer.install() & a13x-quickstart
 - 2.8 Force-Reinstalling & Uninstalling — a13x-uninstall & a13x-purge
 - 2.9 If Maya Can't Find a13x — Automatic Site-Packages Search
- 3. Editing the Scripts — Per-Line Customization Guide
 - 3.1 Logger (logger.py)
 - 3.2 GeoMaster (geomaster.py)
 - 3.3 Revention Autosave (revention.py)
 - 3.4 Substance Loader (substance_loader.py)
 - 3.5 ThumbTac (thumbtac.py)
 - 3.6 Umbra (umbra.py)
 - 3.7 Nomenclator (nomenclator.py)
- 4. Understanding the Apache License 2.0
- 5. Distribution & Forking — What You Can and Can't Do

1. Overview of the Tool Suite

The A13x pipeline is a set of independent Maya utilities that share a common packaging convention: each tool is a single, self-contained Python module that exposes a `show_ui()` (or equivalent) entry point, and each file only launches its UI when run directly — not simply when imported. This lets a shelf button, menu item, or plug-in loader import a module quietly and decide when to pop the window, instead of the window appearing as a side effect of loading the tool.

Tool	File	What it does
GeoMaster	geomaster.py	Scene-wide geometry, topology, UV, shader, and naming checker with one-click fixes, organized into 11 categories.
Revention Autosave	revention.py	Background autosave utility. Saves versioned Maya ASCII files (e.g. <code>asset_RIG_V004.ma</code>) on a timer.
Substance Loader	substance_loader.py	Drag-and-drop texture loader for Arnold; detects texture type from filename and sets the correct color space automatically.
ThumbTac	thumbtac.py	Creates a temporary grid of pivot/snap locators over a selected object — modeling tool only.
Umbra	umbra.py	Qt-based panel of character-model validation checks (color sets, UV sets, influences, history, transforms, joints).
Nomenclator	nomenclator.py	Batch-renames a selected hierarchy into the studio naming convention (<code>_GRP</code> , <code>_SGRP</code> , <code>_GEO</code> , <code>_LOC</code>).
Logger	logger.py	Shared logging helper used internally by GeoMaster; not launched directly by the artist.

Each tool is independent — you can install any subset of them. GeoMaster is the only tool that looks for the shared `logger.py` (and an optional `config.py`); the rest are single-file and have no cross-dependencies.

2. Setup & Installation

There are two ways to get A13x into Maya: a manual copy of the individual `.py` files (Sections 2.1–2.3), or installing the packaged `a13x` distribution from PyPI with `pip` (Section 2.4 onward) — the recommended route, since it also gives you Plug-in Manager integration and a single pinned shelf.

2.1 Manual copy — Step 1: Copy the files

Place the `.py` files in a folder Maya can see — either your user scripts directory (`Documents/maya/<version>/scripts`) or any folder you add to Maya's Python path yourself. GeoMaster expects `logger.py` to sit in the same folder as `geomaster.py`, since it imports it by name.

2.2 Manual copy — Step 2: Launch from the Script Editor

Each tool is opened the same way — import the module, then call its UI function:

Tool	Command to run
GeoMaster	<pre>import sys sys.path.append(r"C:/gameGeoMaster") import geomaster geomaster.show_ui()</pre>
Revention	<pre>import revention revention.show_ui()</pre>
Substance Loader	<pre>import substance_loader substance_loader.create_drag_drop_ui()</pre>
ThumbTac	<pre>import thumbtac thumbtac.show_ui()</pre>
Umbra	<pre>import umbra umbra.show_umbra()</pre>
Nomenclator	<pre>import nomenclator nomenclator.show_ui()</pre>

For a permanent shelf button, put the same two or three lines into a Python shelf button instead of the Script Editor — the behavior is identical.

2.3 Manual copy — Step 3: Optional config.py (GeoMaster only)

GeoMaster and the shared logger will look for a `config.py` file in the same folder. It is entirely optional — both files fall back to sensible built-in defaults if it isn't present. Create one only if you want to override the defaults below:

Setting	Default	Effect
ENABLE_LOGGING	True	Turns the log file and console logging on/off.
LOG_FILE	'gameGeoMaster.log'	Name of the log file written next to logger.py.
DEBUG_MODE	True	When True, log lines are also printed to the Script Editor.
UI_WINDOW_TITLE	'GeoMaster - Geometry Checker'	Title bar text for the GeoMaster window.
UI_WIDTH	600	Default window width in pixels.
UI_HEIGHT	850	Default window height in pixels.

2.2a Section 2.2 Fix — Every Command Now Works on Every Maya Version

Two of the six commands in Section 2.2's table previously only worked on Maya 2022 and earlier, and would fail with an import error on Maya 2025+. This has been fixed - all six now work unchanged, on any supported Maya version, with no extra steps.

What was wrong

GeoMaster already auto-detected PySide6 (Maya 2025+) vs. PySide2 (Maya 2022 and earlier). Revention and Umbra did not - Umbra imported PySide2 with no fallback at all, and Revention fell back only to the legacy pre-2018 PySide/shiboken, never to PySide6. Both raised `ModuleNotFoundError` on a PySide6-only Maya install.

What changed

revention.py and umbra.py now use the same PySide6-first, PySide2-fallback detection as geomaster.py. Both were re-verified after the change with PySide6 as the only binding present - each imports cleanly and its `show_ui()` / `show_umbra()` entry point is callable, the same way GeoMaster's already was. No change to the commands themselves in the Section 2.2 table - the same two or three lines shown there work as-is.

Where else this applies

The Requirements note near Section 2.4 ("GeoMaster auto-detects which one is installed") now applies equally to Revention and Umbra. ThumbTac, Substance Loader, and Nomenclator were never affected - they use Maya's native cmds UI only and have no Qt/PySide dependency of any kind.

Requirements: Autodesk Maya with PySide2 (Maya 2022 and earlier) or PySide6 (Maya 2025+). GeoMaster auto-detects which one is installed. Revention, Umbra, and Substance Loader's UI code will raise an import error if run outside Maya — that is expected; they are not meant to run as standalone scripts.

2.4 Installing via pip — TestPyPI & PyPI (Recommended)

A13x is also published as a normal Python package named `a13x`, so it can be installed with pip like any other library — either from **TestPyPI** (a staging index used for verifying a release before it goes public) or from the real, production **PyPI**. Always run pip against `mayapy` — Maya's own bundled Python interpreter — not your system Python, or Maya will not see the package.

Index	Command
PyPI (production)	<code>mayapy -m pip install a13x</code>
TestPyPI (staging)	<code>mayapy -m pip install --index-url https://test.pypi.org/simple/ a13x</code>

Both commands install the exact same package structure — the only difference is which index it comes from. TestPyPI is useful for confirming a new release installs cleanly before publishing the same version to production PyPI.

If more than one Maya version is installed on this machine, each version has its own separate `mayapy` and its own separate site-packages — a pip install against one version's `mayapy` does not reach any other version. The install scripts in the Quick Start page handle this for you automatically, by running the pip install (and the registration in Section 2.6) once per Maya version found, rather than just one.

"mayapy isn't recognized as an internal or external command" means exactly what it says: Windows genuinely couldn't find a program by that name anywhere it looked. This is not something a plain Python install can fix — `mayapy` is Maya's own Python build, compiled with Maya's API baked in, and it only exists as a file inside an actual Autodesk Maya installation. Installing Python from `python.org` gives you a normal interpreter with no `maya.cmds` at all, so it can't substitute for `mayapy`. The install scripts search `PATH`, the default Program Files locations, the `MAYA_LOCATION` environment variable, and (on Windows) the registry — but if Maya was installed somewhere none of those catch, or isn't installed at all, you'll be prompted to paste the exact path. The one place that's always correct: open Maya itself, go to the Script Editor (Python tab), and run `import sys; print(sys.executable)` — that prints `mayapy`'s real location no matter where Maya was installed.

2.5 Where Everything Gets Installed

Installing with pip and registering the plug-in in Maya are two separate steps, and each one writes files to a different place on disk:

Step	Lands in	Details
pip install (Section 2.4)	mayapy's site-packages	The a13x package folder (source modules, thumbnails, the docs folder holding this PDF, and the a13x-quickstart command) is copied into mayapy's own site-packages directory — the same one that command was run against. It is not yet visible inside Maya's UI at this point.
installer.install() (Section 2.6)	Maya's user plug-ins folder	Run automatically by the install scripts right after the pip install (Section 2.4), or once by hand from Maya's Script Editor. Writes a small stub file (<code>a13x_plugin.py</code>) into Maya's plug-ins folder and turns on "Auto load", which is what makes A13x appear as a checkbox entry in Plug-in Manager.

Typical mayapy site-packages paths (pip install target)

OS	Path
Windows	C:\Program Files\Autodesk\Maya<version>\Python\Lib\site-packages\A13x\
macOS	/Applications/Autodesk/maya<version>/Maya.app/Contents/Frameworks/Python.framework/Versions/Current/lib/python3.*/site-packages/A13x/
Linux	/usr/autodesk/maya<version>/lib/python3.*/site-packages/A13x/

Run `python -c "import A13x, os; print(os.path.dirname(A13x.__file__))"` with the same mayapy to print the exact path on your machine instead of guessing the version folder.

Maya user plug-ins folder (installer.install() target)

OS	Path
Windows	%USERPROFILE%\Documents\maya\<version>\plug-ins\
macOS	~/Library/Preferences/Autodesk/maya/<version>/plug-ins/
Linux	~/maya/<version>/plug-ins/

2.6 Loading the Plug-in & Using the Shelf — Autoload Explained

This section only applies to the pip install path (2.4 onward). If you used the manual copy method (2.1–2.3) instead, keep using `import geomaster; geomaster.show_ui()` and skip this section — there is no plug-in or shelf to load in that path.

`installer.install()` (Section 2, Step 2) does two separate things, and it's worth knowing which is which:

What <code>installer.install()</code> does	In detail
Loads it right now, in this session	<code>cmds.loadPlugin(...)</code> runs immediately. This is what actually registers one Maya command per tool (a13xGeoMaster, a13xNomenclator, ...) and builds the A13x shelf, pinned next to your other shelves, with one button per tool.
Turns Autoload on	A separate, persistent Maya setting (<code>cmds.pluginInfo(..., autoload=True)</code>) stored outside the plug-in file. From your NEXT Maya launch onward, Maya loads A13x automatically before you do anything — the Plug-in Manager checkbox is already ticked and the shelf is already there.

In practice: you only ever run `installer.install()` yourself **once**, the very first time. That call is both the initial load for the current session AND the one-time setup that makes every later Maya launch load A13x on its own — you never need to run it again unless you've uninstalled and are setting up a fresh copy.

Registering without opening Maya at all

The install scripts run this same registration step for you automatically, right after the pip install — once for **every** Maya version found on the machine, not just one, so you don't have to type `installer.install()` into each version's Script Editor by hand. This works by briefly starting a temporary, invisible Maya session (mayapy + Maya's own `maya.standalone` module) just long enough to run `installer.install()`, then closing it again — Maya's actual application window never opens.

You can also trigger this directly, any time, for whichever mayapy you point it at, without running the full install script again:

Goal	Command
Register the plug-in headlessly, from a terminal	<code>a13x-register-plugin</code>

Command registration and shelf-building behave differently here, and that's expected. Registering the six Maya commands (a13xGeoMaster, etc.) and turning Autoload on both work fully in this headless session. Building the A13x shelf does not — it needs Maya's actual shelf UI to attach to, which doesn't exist in a headless session, so `a13x-register-plugin` skips it with a printed note rather than erroring. Since Autoload was just turned on regardless, the shelf builds itself the moment you next open Maya normally — you just won't see it appear from the terminal directly.

Using the tools day to day

Once loaded, forget about imports entirely — click the buttons on the **A13x** shelf. Each button runs one of the Maya commands the plug-in registered on load:

Tool	Maya command (equivalent to the shelf button)
GeoMaster	a13xGeoMaster
Nomenclator	a13xNomenclator
ThumbTac	a13xThumbTac
Revention	a13xRevention
Substance Loader	a13xSubstanceLoader
Umbra	a13xUmbra

The command column is only there for scripting a tool launch from your own code (a hotkey, another shelf, a startup script) — day to day, the shelf button is all you need; you don't type these by hand.

Checking or changing autoload status

Confirm or change this any time in Windows > Settings/Preferences > Plug-in Manager — find `a13x_plugin.py` in the list. Two independent checkboxes sit next to it: **Loaded** (is it active right now) and **Auto load** (should Maya load it automatically on every future launch). Unticking Auto load there stops future auto-loading without uninstalling anything — untick Loaded too if you want it fully inactive for the rest of this session as well.

If a fresh Maya launch does *not* show the A13x shelf even though Auto load is ticked, that's usually Maya failing to re-import the package on startup — most often because it was pip-installed with a different Python than the mayapy that launches Maya. Section 2.9's automatic search runs exactly at this point and should recover it on its own; if the shelf still doesn't appear, re-run `installer.install()` once from the Script Editor to confirm.

2.7 Opening This Guide — `installer.install()` & `a13x-quickstart`

`pip` does not run any code automatically as part of installing a package — that's true for every `pip` install, from `TestPyPI` or `PyPI` alike, and is a deliberate security property of the wheel format, not something specific to `A13x`. So the guide can't pop up as a literal side effect of the `pip` install command itself. Instead, `A13x` opens it automatically at the next available moment: the end of the Maya-side `installer.install()` call from Section 2, Step 2 above.

When	How
Inside Maya, once per machine (recommended)	<code>installer.install()</code> opens the guide automatically after registering the plug-in — no extra command needed.
From a terminal, any time, without opening Maya	Run <code>a13x-quickstart</code> , installed alongside the package.

Both open the exact same PDF, located the same way: relative to wherever `pip` actually installed the `a13x` package on this machine (Windows, macOS, or Linux) — see Section 2.5 for those paths. Neither one requires you to know or type that path yourself.

If `installer.install()` registers the plug-in successfully but the PDF doesn't visibly open, check the Script Editor's output — `A13x` prints the exact path it tried instead of failing silently, and most often this means the machine simply has no default PDF viewer associated with `.pdf` files yet.

2.8 Force-Reinstalling & Uninstalling Cleanly

Why the old `.bat` / `.sh` install scripts never showed up after ``pip install a13x``: they lived in this project's `install/` folder, which sits outside the `a13x` package itself. A wheel built from this project only ever contains what's inside the package — everything under `src/a13x/` — so those scripts were never part of what got uploaded to `PyPI` or `TestPyPI` in the first place. It was never a broken download; there was nothing there to download. The fix below moves that functionality inside the package as console commands instead, so it's actually shipped by `pip`.

Ordinary upgrades (`mayapy -m pip install --upgrade a13x`) are enough for most version bumps. Occasionally — after a version was pulled and re-pushed under the same number on `TestPyPI`, or if a previous install left files behind — `pip` needs to be told explicitly not to trust anything it has cached locally, and not to skip files it thinks are already correct:

Goal	Command
Force a clean reinstall (<code>PyPI</code>)	<code>mayapy -m pip install --upgrade --force-reinstall --no-cache-dir a13x</code>
Force a clean reinstall (<code>TestPyPI</code>)	<code>mayapy -m pip install --upgrade --force-reinstall --no-cache-dir --index-url https://test.pypi.org/simple/ a13x</code>

- `--force-reinstall` makes pip overwrite every file even if it believes the version already matches;
 - `--no-cache-dir` makes it re-download the wheel instead of reusing a locally cached copy from an earlier attempt. Use both together whenever you suspect a stale install rather than a genuinely new version.

Uninstalling — the `a13x-uninstall` command

pip automatically creates a small launcher for every command listed under a package's `[project.scripts]` the moment it installs that package — this is standard pip behavior, works identically on Windows, macOS, and Linux, and needs no separate script file to still exist anywhere. A13x registers four such commands, so all four are guaranteed present right after `pip install a13x`, from either index:

Command	What it does
<code>a13x-quickstart</code>	Opens this PDF guide.
<code>a13x-register-plugin</code>	Registers the plug-in in Maya headlessly (see Section 2.6) — no need to open Maya's UI or paste anything into the Script Editor.
<code>a13x-uninstall</code>	Removes the Maya plug-in stub, uninstalls the pip package via the exact interpreter running the command, then sweeps for stray copies (see Section 2.9).
<code>a13x-purge</code>	Lists — or with <code>--purge</code> , removes — every a13x install found anywhere on the machine, regardless of which interpreter installed it.

Run `a13x-uninstall` from any terminal — no Maya, no need to know or match which Python originally ran the install:

Goal	Command
Fully uninstall (Maya stub + pip package + stray copies)	<code>a13x-uninstall</code>
Just see where every copy is, without removing anything	<code>a13x-purge</code>
Preview exactly what a purge would delete first	<code>a13x-purge --dry-run</code>

"pip uninstall says a13x isn't installed" almost always means a DIFFERENT Python installed it. pip only ever looks inside the one interpreter it's currently running under. `a13x-uninstall` sidesteps this: step 2 always uses the exact interpreter that installed the command itself (there's no other way it could be running), and step 3 sweeps every other known Python layout — including python.org's newer per-user installer folders such as `pythoncore-3.14-64` — for anything left behind by an earlier, mismatched install. See Section 2.9 for the full location list.

2.9 If Maya Can't Find a13x — Automatic Site-Packages Search

Section 2.4 recommends always running pip against mayapy specifically, since that's the Python interpreter Maya itself uses. In practice, though, it's easy to pip install with the wrong interpreter by accident — a system Python, a different Maya version's mayapy, or a virtualenv — and end up with a13x sitting in site-packages somewhere Maya doesn't automatically look.

To make that recoverable without a manual reinstall, the Maya plug-in stub written by `installer.install()`, the standalone `a13x.pathfinder` module, and the `a13x-purge` console command all search the same exhaustive list of pip install locations across Windows, macOS, and Linux before giving up — not just Maya's own layout:

Layout	Locations searched
Windows — Maya	...\Autodesk\Maya*\Python(3)\Lib\site-packages
Windows — python.org (current, per-user)	%LOCALAPPDATA%\Python\pythoncore-3*-64\Lib\site-packages (e.g. pythoncore-3.14-64)
Windows — python.org (older, per-user)	%LOCALAPPDATA%\Programs\Python\Python3*\Lib\site-packages, %APPDATA%\Python\Python3*\site-packages
Windows — system-wide / Store	...\Program Files\Python3*\Lib\site-packages, C:\Python3*\Lib\site-packages, Windows Store Python
macOS — Maya / python.org / Homebrew	...Maya.app/.../Python.framework/..., ~/Library/Python/3*/..., /Library/Frameworks/Python.framework/..., /opt/homebrew/lib/python3*/site-packages
macOS / Linux — conda & pyenv	~/anaconda3, ~/miniconda3, ~/miniforge3 (base + every env), ~/pyenv/versions/*/...
Linux — Maya / system	/usr/autodesk/maya*/..., /usr/lib/python3*/site-packages, /usr/lib/python3/dist-packages, /usr/local/lib/python3*/...
All platforms	~/.local/lib/python3*/site-packages, any ~/.virtualenvs/*/... or ~/.venv/...

The first matching folder that actually contains an a13x subfolder is added to `sys.path` automatically, and the import is retried — so the plug-in still loads even though pip and mayapy didn't originally agree on where it should go. If nothing is found anywhere, the Script Editor prints the exact `mayapy -m pip install a13x` command to fix it, rather than failing silently — and `a13x-purge` (no arguments, from any terminal) prints this same full list with a per-location result, so "not found" is never a mystery.

This search is a safety net, not a substitute for Section 2.4's advice. Installing with the correct mayapy the first time is still faster and more predictable — this just means a mismatched install isn't a dead end, and cleaning one up doesn't depend on first figuring out by hand which of a dozen possible Python installs it landed in.

3. Editing the Scripts — Per-Line Customization Guide

None of these tools require code changes to run. The tables below point out the specific lines a technical artist is most likely to want to tweak — naming conventions, default numbers, colors, and thresholds — without having to read the full source. Open the file in a text editor, use Find (Ctrl/Cmd+F) to jump to the variable name, and change the value on that line only.

3.1 Logger

File: `logger.py`

Internal helper used by GeoMaster. Most artists won't need to touch this — use `config.py` instead (see Section 2.3). Direct edits here change the fallback behavior used when no `config.py` exists.

Line(s)	What to edit	Effect of changing it
12–16	The class <code>config: fallback block (ENABLE_LOGGING, LOG_FILE, DEBUG_MODE)</code>	Changes the default logging behavior when no <code>config.py</code> is present in the folder.
~50	<code>log_line = "[%s] [%s] %s\n"</code>	Changes the format of each line written to the log file.

3.2 GeoMaster

File: `geomaster.py`

The largest tool in the suite (~3,100 lines). The checks are grouped into numbered categories; most tunable values are function default-arguments near the top of each category block.

Line(s)	What to edit	Effect of changing it
69–75	<code>config / logger import block</code>	Controls what happens if <code>config.py</code> or <code>logger.py</code> can't be found — GeoMaster still runs, just without logging.
~727	<code>find_non_planar_faces(tolerance=0.001)</code>	Raise the tolerance to flag fewer near-planar faces as errors; lower it to be stricter.
~789	<code>find_duplicate_vertices(threshold=0.0001)</code>	Distance under which two vertices count as duplicates.
~842	<code>find_zero_length_edges(threshold=0.0001)</code>	Minimum edge length before it's flagged as zero-length.
2307–2309	<code>config.UI_WINDOW_TITLE / UI_WIDTH / UI_HEIGHT</code>	Window title and default size — override via <code>config.py</code> rather than editing this block directly.

3.3 Revention Autosave

File: `revention.py`

Controls what suffix choices appear in the dropdown and the default save interval.

Line(s)	What to edit	Effect of changing it
209	<code>self.suffix_combo.addItem(['RIG', 'LDV', 'MDL', 'GRM', 'SIM', 'ANM', 'SKN', 'WGT', 'BLD'])</code>	Add, remove, or reorder the suffix tags available in the UI (used in filenames like <code>asset_RIG_V004.ma</code>).
217–219	<code>interval_spin min / max / default (1, 1440, 15)</code>	Sets the allowed range and the default autosave interval, in minutes.
~55	Versioned filename pattern, <code>base_SUFFIX_V###.ma</code>	Change the padding or separator used when numbering saved versions.

3.4 Substance Loader

File: `substance_loader.py`

Texture-type detection and color-space assignment are both driven by two small lookup tables near the top of the file — edit these rather than the functions below them.

Line(s)	What to edit	Effect of changing it
17–25	<code>COLORSPACE_CONFIG</code> dict (diffuse/basecolor/albedo → sRGB, roughness/metalness/normal → Raw)	Which Arnold color space gets applied per texture type on load.
29–34	<code>TEXTURE_SLOTS</code> list (diffuse, roughness, metalness, normal)	Which texture slots appear in the UI, and their display labels. Add a tuple here to add a new slot.
36	<code>UDIM_TOKEN = '<UDIM>'</code>	The placeholder token substituted into UDIM filenames.

3.5 ThumbTac

File: `thumbtac.py`

This one is explicitly written with a "CONFIG (edit these)" block — the file's own header points here first.

Line(s)	What to edit	Effect of changing it
34	DEFAULT_DIVISIONS = 3	Default grid density per axis (3 = a 3×3×3 grid of locators).
36	NODE_SCALE_FACTOR = 0.02	Locator size as a fraction of the object's bounding-box diagonal.
38	MIN_NODE_SIZE = 0.001	Floor value so locators never shrink to zero on tiny objects.
40	CREATE_BUTTON_COLOR = (0.3, 0.5, 0.9)	RGB (0–1 range) color of the Create button.
18	WINDOW_NAME = 'thumbtacWindow'	Internal window ID — change only if it clashes with another tool's window name.

3.6 Umbra

File: `umbra.py`

Each validation check is a button added to a list. To add, remove, or relabel a check, edit this block — you don't need to touch the check functions themselves.

Line(s)	What to edit	Effect of changing it
157–186	<code>self._check_buttons.append(UmbraButton(...))</code> block – 8 entries	Each entry is one button: its label, grid position (row, col), and which method it calls. Copy a block and point it at a new method to add a check; delete a block to remove one.
~18	<code>class UmbraButton</code> constructor	Change button styling (blue background) here to re-theme every check button at once.

3.7 Nomenclator

File: `nomenclator.py`

The naming convention is defined once at the top of the file as a list of suffixes; the classification logic below reads from it.

Line(s)	What to edit	Effect of changing it
20	SUFFIXES = ['_GRP', '_SGRP', '_GEO', '_LOC']	The known suffixes that get stripped before a new one is applied. Add a suffix here if you introduce a new node category.

Line(s)	What to edit	Effect of changing it
52–62	<code>classify()</code> – checks for mesh / locator shape types	Add another if <code>shape_type == ...</code> branch to recognize other shape types (e.g. NURBS curves) and give them their own suffix.
~80	Uppercasing rule: <code>new_name = base_name.upper() + '_LOC'</code>	Locators are forced to UPPERCASE; remove <code>.upper()</code> to keep the artist's original casing.
166–198	<code>show_ui()</code> – <code>dark_blue</code> / <code>accent_blue</code> tuples	Window colors for the tool's UI.

Line numbers marked with "~" are approximate and can shift slightly between versions of a file — use Find to search for the variable name shown rather than jumping to the exact line.

4. Understanding the Apache License 2.0

If the A13x pipeline is distributed under the Apache License, Version 2.0, here is what that means in plain terms for anyone installing, modifying, or passing the tools along.

What it is

The Apache License 2.0 is a permissive open-source software license published by the Apache Software Foundation. "Permissive" means it grants very broad rights to use, modify, and redistribute the code — including in closed-source, commercial products — with only a small set of conditions attached. It stands in contrast to "copyleft" licenses (like the GPL), which force derivative works to be released under the same license.

The three things every permissive license balances

Category	Includes	In practice
Permissions	Commercial use · Modification · Distribution · Sublicensing · Private use · Patent use	You can use the tools at a studio, sell a product built on them, change the code, and share your changes — or keep them internal.
Conditions	License & copyright notice · State changes · Preserve notices	Any copy you distribute must include the original license text and copyright notice, and you must note that you changed a file if you modified it.
Limitations	No liability · No warranty · No trademark grant	The software is provided "as is." The authors aren't liable if something breaks, and you don't get rights to use the original author's name, logo, or trademarks.

A patent grant, with a catch

Unlike some older permissive licenses, Apache 2.0 includes an explicit grant of patent rights from every contributor covering their own contributions. That protection has one condition built in: if you sue someone claiming the software infringes a patent, your own patent license under this grant terminates automatically. It's designed to discourage patent litigation between people building on the same codebase.

5. Distribution & Forking — What You Can and Can't Do

Sharing the tools as-is

You're free to hand the scripts to another artist, another studio, or post them publicly — including publishing the packaged version to PyPI or TestPyPI, in source or compiled form, for free or for money. The only requirement is to carry the license along with the code — typically a LICENSE (or NOTICE) file in the same package, already bundled into the wheel and sdist — and to keep any existing copyright notices intact rather than stripping them out.

Forking or modifying the tools

Forking — taking a copy of the repository or files and continuing development on your own, independent of the original — is explicitly allowed. If you distribute your modified version (rather than keeping it purely for internal studio use), Apache 2.0 asks for two things:

- A note that you changed the files (a line in a changelog, README, or the file header is enough — there's no prescribed format).
- The original license and copyright notice still travel with the code, even though your changes and any new copyright notice for your own contributions can sit alongside it.

You do not need permission from the original author to fork, and you can relicense your own additions under different terms if you want — but the portions you received under Apache 2.0 remain under Apache 2.0. You also cannot use the original project's name, logo, or branding to imply your fork is officially endorsed, since trademarks are explicitly not part of the grant.

Internal studio use

None of the above is triggered by simply installing and using the tools internally at a studio without redistributing them further — the notice and "state changes" requirements apply when you pass copies (modified or not) to someone outside your organization.

Quick checklist before you redistribute a modified copy

	Item
✓	Keep the original LICENSE / NOTICE file in the package.
✓	Keep existing copyright headers in files you didn't touch.
✓	Add a short note where you changed a file (header comment or changelog entry).
✓	Don't use the original tool's name/branding to suggest official endorsement.
–	No need to open-source your own additions — that's your choice.
–	No need to ask permission first — the license is the permission.

This section is a plain-language summary for a technical audience, not legal advice. If a redistribution decision has real business stakes — selling a product, bundling with other licensed code, a client deliverable — have the actual license text reviewed by counsel before you rely on this summary.